Systems Administration (ASIST)

Practical Theoretical Class 04

Network configuration management.
DHCP servers. DNS servers.

Configuring Network Interfaces on Servers

The configuration of a network interface can be done autonomously by the system itself or using external services. The most commonly used external service for this effect is *Dynamic Host Configuration Protocol* (**DHCP**), in the case of IPv6 configuration you can also use *Stateless Address Auto Configuration* (**SLAAC**), possibly supplemented with **DHCP**.

As a server typically provides network services, it must have fixed addresses known to clients. This excludes the use of SLAAC, but DHCP can be used to assign fixed IPv4 and IPv6 addresses.

As already mentioned, a server must be independent of external services. For this reason, we typically opt for standalone network configurations where the system has all the data to configure its network interfaces. Thus, even if the network's DHCP services are not operational, the server can function normally.

From the point of view of the IPv4 and IPv6 protocols, to configure a network interface is enough two data: **node address** and **network prefix** (netmask).

With these two elements it is possible to deduce which is the **network address**, and in the case of IPv4, which is the **broadcast address**.

Network interface names

In the operating system each *network interface controller* (**NIC**) is identified by a name. If the server has multiple network interfaces it is critical to identify the match between the names and the physical interfaces, this may not be trivial, especially if the interface hardware is the same.

Linux: traditionally Ethernet network interfaces were identified by eth followed by a numeric suffix (***eth0***, ***eth1***, ***eth2***,…). In current *kernel* versions, Ethernet network interfaces are identified by **en** followed by a complex suffix representing their hardware positioning. For example **ens**… means this is an interface connected to the external PCI bus, and e**no**… it means that it is an on-board interface. The rest of the prefix is also hardware related.

Windows: network interfaces are identified with names that try to represent their type, for example, **Ethernet**. If there are multiple interfaces of the same type, the first one retains the name and the next ones are given a numeric suffix from 2. For example on a server with 3 Ethernet interfaces: **Ethernet**, **Ethernet 2**, and **Ethernet 3**. IPv4 and IPv6 configuration of graphical mode Windows Server interfaces is easily done using ***Control Panel***.

Linux - configuration of network interfaces

The **ifconfig** and **ip** commands can be used to view the current state of network interfaces and change their IP settings. Note however that changes made in this way do not persist after a ***reboot*** of the machine (unlike changes made in Windows via ***Control Panel***).

Network settings applied during machine ***boot*** reside in configuration files that depend on distribution and even version; on Debian and Ubuntu 16.04 are in the file **/etc/network/interfaces** and on Ubuntu 18.04 as network interfaces are managed by **Netplan** are in the file **/etc/netplan/50-cloud-init.yaml**.

The **ifconfig** command without arguments displays a list of active (*up*) network interfaces and their settings. The **ip addr show** command displays a list of all network interfaces and their settings (*up* or *down*).

The **ip** command should be used instead of the **ifconfig** command that is expected to be discontinued in the future.

Basic example of configuring a new interface with an IPv6 address followed by interface activation (assuming it was **down**):

```
ip address add 2001:0db8:85a3::0370:7334/64 dev ens160
ip link set dev ens160 up
```

Default gateway and routing

The node address and network prefix are sufficient to be able to communicate with other nodes of the IP networks to which the server is directly connected, however, to communicate with nodes of other IP networks it is necessary to use ***routers***.

In the simplest scenario, in the IP networks to which the server is connected there is only one router, in which case just set it as ***default gateway***. In case of Windows systems the ***default gateway*** can be set together with IP settings. On a Linux system it must be manually added to the system ***routing table*** or set in the network configuration.

As a result, all packets destined for remote IP networks (to which the server is not directly attached) will be transmitted to the ***default gateway***.

The IPv4 and IPv6 ***default gateway*** configurations are completely independent, so if the server uses both IP4 and IPv6 (*dual-stack*) there will be an IPv4 ***default gateway*** and an IPv6 ***default gateway***; they can be the same, but the addresses will certainly be different.

In a scenario where there are multiple routers on the IP networks to which the server is attached, the ***routing table*** must be set accordingly.

Linux – *routing table* management

The traditional **route** command is expected to be discontinued and the **ip [-6] route** command should be used instead. The **-6** option is used to manage the IPv6 *routing table*, if missing it acts on the IPv4 *routing table*.

We can view the tables using the command **ip [-6] route show**

Again, configuration changes do not persist after a reboot, so they must be set in appropriate distribution dependent configuration files.

Examples:

- IPv4 and IPv6 ***Default gateway*** definition on NIC ens160:

```
ip route add 0.0.0.0/0 via 10.20.0.1 dev ens160
ip -6 route add ::/0 via fd1e:2bae:c6fd:627a::55 dev ens192 protocol static
```

- Adding a specific rule in IPv4 and IPv6 on NIC ens160:

```
ip route add 172.16.5.0/24 via 10.20.0.1 dev ens160
ip -6 route add fd1e:2bae::/64 via fd1e:2bae:c6::5 dev ens192 protocol static
```

- Removing a rule on IPv4 and IPv6 on NIC ens160:

```
ip route del 172.16.5.0/24 via 10.20.0.1 dev ens160
ip -6 route del ::/0 via fd1e:2bae:c6fd:627a::55 dev ens192 protocol static
```

Windows – *routing table* management

Routing table management beyond the default gateway setting must use the **route** command on the command line. The **route print** command displays the current tables (IPv4 and IPv6).

Configuration changes are persistently saved by the operating system.

Depending on the addresses provided, the command deduces whether these are IPv4 or IPv6 addresses, and also determines the most appropriate interface.

Examples (equivalent to previous ones on Linux):

- IPv4 and IPv6 ***Default gateway*** definition:

```
route add 0.0.0.0/0 10.20.0.1
route add ::/0 fd1e:2bae:c6fd:627a::55
```

- Adding a specific rule in IPv4 and IPv6:

```
route add 172.16.5.0/24 10.20.0.1
route add fd1e:2bae::/64 fd1e:2bae:c6::5
```

- Removing a rule on IPv4 and IPv6:

```
route delete 172.16.5.0/24
route delete ::/0
```

If desired, you can enter the desired network interface with the **-F** option.

DNS client configuration (Name Resolution))

To be able to use DNS system hostnames instead of IP addresses it is necessary to define **which DNS name server addresses to use.**

The DNS system only resolves qualified names (which include the full domain name), to resolve unqualified names several domain names can be defined that will be used to fill in the unqualified names before sending them to the DNS server. In a system that uses DHCP this information is normally transmitted by this way, not being used DHCP will have to be set manually.

Windows: DNS client configuration is performed together with the IP address settings of the network interfaces. Although in the base window only one or two DNS servers can be defined, in the advanced options it is possible to add more. Also in the advanced options it is possible to define a set of domain names to use to complete unqualified names.

Linux: DNS client configuration (*resolver*) is traditionally saved in the file **/etc/resolv.conf**. However, in most current distributions this file is generated every time the machine is booted, for this purpose information residing in other configuration files (depending on the distribution is used). For Ubuntu 16.04 or 18.04 this information should be set in the configuration file of the network interface(s).

Network Interfaces with multiple IP addresses

The same network interface can be assigned multiple IP addresses, which may belong to different IP networks or may belong to the same IP network.

It is possible to have several different IP networks operating on the same physical network. IP nodes that, although connected to the same physical network, use addresses from different IP networks can only communicate with each other through a *router*. The need to resort to the router can be avoided if addresses have been assigned to the interface on each of the networks.

We can also assign to the same interface several IP addresses belonging to the same IP network, in this case the goal is usually to create several apparent servers (virtual).

If the server has multiple IP addresses (typically mapped to distinct DNS names), from the client's point of view it will appear to be a set of several distinct servers.

For this type of configuration to become interesting it must be complemented with a network services configuration that is dependent on the destination address of the client requests. For example, the Apache HTTP server allows you to define distinct virtual sites based on this criterion.

Windows: in the advanced options of IPv4 and IPv6 properties of the network interface, other IP addresses can be added to what was defined in the base window. In this case, for addresses belonging to different networks it will not be possible to define a default gateway for each one of them, this configuration must be done using the **route** command.

Linux: using the **ip addr add**… command we can add several addresses to the interface besides the first one (although they only persist until the next system *boot*).

By adding an address we can also define a virtual interface name, for compatibility with *Linux-2.0 net aliases*, the name must start with the physical interface name followed by a colon*, the rest can be defined according to administrator preferences. Example:

```
root@uvm:~# ip addr add 10.5.10.200/16 broadcast + dev ens160 label ens160:rede1
root@uvm:~# ifconfig ens160:rede1
ens160:rede1 Link encap:Ethernet  Endereço de HW 00:50:56:a7:10:03
      inet end.: 10.5.10.200  Bcast:10.5.255.255  Masc:255.255.0.0
      UP BROADCAST RUNNING MULTICAST  MTU:1500  Métrica:1
```

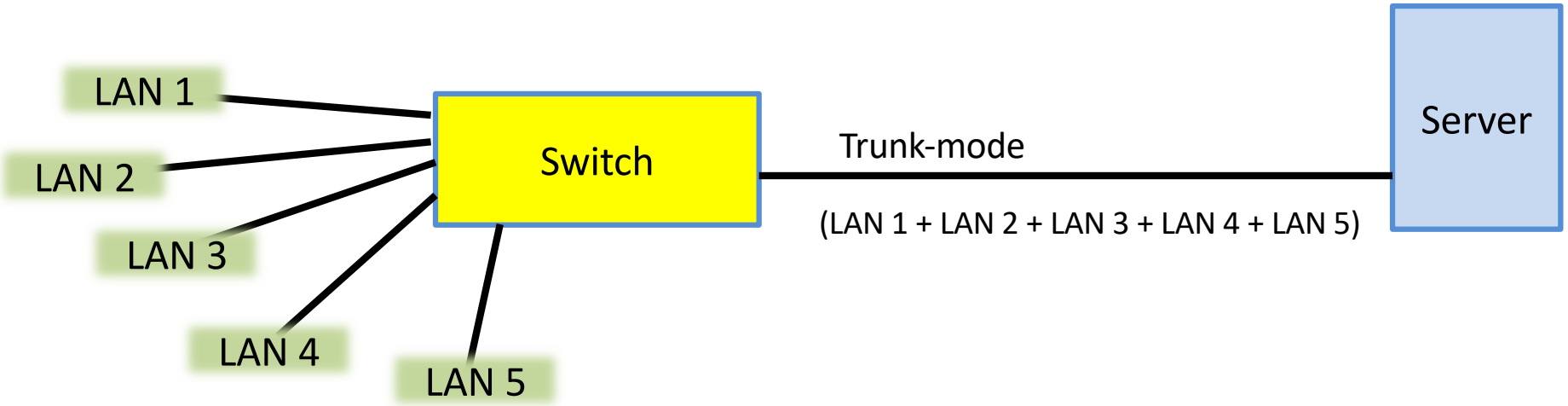
As can be seen from the result of the **ifconfig** command, the network interface named **ens160: rede1** came into existence on the system, this facilitates the configuration of network services because most of them allow you to explain which network interface to use..

* The colon is a convention, not an imposition

VLANs configuration

By using IEEE 802.1q VLAN tags (**VLANID**) it is possible to use a single network interface to connect to a large number of distinct physical networks, this is the typical use of VLANs on servers.

This will require a ***switch*** connected to the various physical networks and establishing a VLANID for each of them. The gateway to the server is configured in trunk mode, so traffic from the various physical networks will be sent through it properly labeled with VLANID.



Thanks to the use of tags at both ends of the server connection, traffic from different physical networks will not mix. On the server side, on the network interface, a **VLAN interface** must be defined for each of the physical networks using the VLANIDs set on the switch.

VLANs configuration on Windows Server

Recent versions of Windows Server use ***NIC Teaming*** to define multiple VLANs on a network interface using IEEE 802.1q tags..

The ***NIC Teaming*** technique is used to group multiple physical interfaces into a single logical interface (***NIC Team***), the goal is to increase **load-balancing** and ensure **failover**.

NIC Teaming is also used to define multiple VLANs connected to the same physical interface, but in this case the *NIC Team* consists of only one physical interface.

VLAN interfaces can then be added to the *NIC Team* by defining their VLANID and assigning appropriate names to each new VLAN interface. Note that when adding a physical interface to a *NIC Team*, the physical interface IP configuration is no longer used, the IP configuration must be set again on the *NIC Team* interface via the ***Control Panel***.

Once VLANs have been added to the *NIC Team*, the IP configuration of each VLAN interface is also performed as usual in the ***Control Panel***.

NIC Team interface *(primary interface)* do not use VLANID tags (*untagged frames)*, VLAN interfaces use tag (*tagged frames*).

VLAN configuration on Linux

To create VLAN interfaces over a physical interface use the command **vconfig**:

vconfig add INTERFACE VLANID

The command creates a VLAN interface on the physical interface INTERFACE where the VLAN tag provided as the second argument (VLANID) will be used. The new VLAN interface will be named **INTERFACE.VLANID** (in fact the way to name the VLAN interfaces can be set using the **vconfig set_name_type FORM-NAME** command that will apply to interfaces added later; the default value of FORM-NAME is **DEV_PLUS_VID_NO_PAD** which corresponds to **INTERFACE.VLANID**).

Once you have added the VLAN interface, your IP settings can be configured using the **ip addr add**… command.

The physical interface (INTERFACE) continues to operate normally without VLANID tags (*untagged frames*). VLAN interfaces use tags (*tagged frames*).

The option **rem** of **vconfig** removes a VLAN interface:

vconfig rem VLAN-INTERFACE

Normally: vconfig rem INTERFACE.VLANID

VLANs configuration on Linux - example

In this example two VLAN interfaces are created over the ens160 physical interface with **300** and **320** tags, IPv4 addresses are assigned to all interfaces, and finally the interfaces are enabled:

```
vconfig add ens160 300

vconfig add ens160 320

ip addr add 192.168.5.1/24 broadcast + dev ens160

ip addr add 192.168.6.1/24 broadcast + dev ens160.300

ip addr add 192.168.7.1/24 broadcast + dev ens160.320

ip link set dev ens160 up

ip link set dev ens160.300 up

ip link set dev ens160.320 up
```

Packets issued on **ens160** interface do not contain VLAN tags, packets issued on **ens160.300** interface have VLAN tag 300, packets issued on **ens160.320** interface have VLAN tag 320.

On the **ens160** interface only untagged packets are received, on the **ens160.300** interface only tagged packets 300 are received, on the **ens160.320** interface only tagged packets are received 320. Packets received with other tags will be discarded.

VLANs vs. Multiple addresses on same interface

At first glance creating multiple VLAN interfaces over the same physical interface or assigning multiple IP addresses to the same physical interface may seem similar solutions.

In either case we may have IP packets belonging to different IP networks circulating on the same physical network. The difference is in the way packets from different IP networks are distinguished from each other.

Not using VLANs, the distinction is made by IP addresses, so at level 3 this has some consequences. For example, packets sent to *multicast* or generic *broadcast* addresses (255.255.255.255) cannot determine to which network they belong. This may impact protocols using this type of address.

Another drawback is that without using VLANs it will not be possible to connect multiple networks to a switch because it operates at level 2 and does not parse IP addresses.

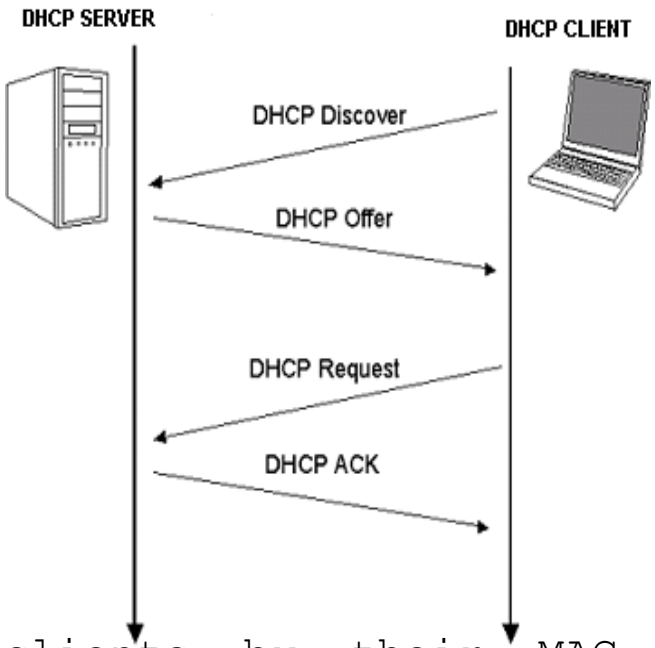
When using VLANs, the distinction is made by the VLANID carried in the frames, therefore at level 2. Each VLAN is for all intents and purposes a distinct physical network, a frame issued in one VLAN never under any circumstances propagates to a different VLAN.

Namely, IP packets sent to broadcast and multicast addresses never pass from one VLAN to another VLAN.

DHCP service (DHCP server)

While configuring server network interfaces through DHCP is advised against, for workstations it is critical. Allows a user when connecting their workstation to an unfamiliar network (wired or wireless) to obtain the appropriate IP configuration data for that network. For example on Linux systems the **ifplugd** service detects when a network cable is connected to the equipment and invokes the services needed to configure the network interface via DHCP.

The DHCP service uses UDP packets, the client's first request is sent to the generic broadcast address **255.255.255.255** for IPv4 or the multicast link local address **ff02::1:2** for IPv6 (**DHCPv6**). The image illustrates the various steps in the case of IPv4, for DHCPv6 the scenario is similar although the messages are different.



Using link-local broadcast or multicast implies that on the physical network there must be at least one DHCP server.

Usually DHCP servers identify the various clients by their MAC addresses, their main mission is to provide each client with unique IP addresses on the network. For this purpose they manage the ranges of addresses available on the network.

In addition to the unique IP address for each client, the DHCP server provides or may provide other parameters that are normally identical for all clients on the same network: network maskm, *default gateway*, DNS servers addresses and the local DNS domain name (to complete unqualified names). In addition to these parameters, DHCP responses can also contain many others for various purposes.

DHCP service can be provided by many types of network equipment such as routers, Layer 3 switches and servers.

Windows Server: DHCP Server role can be activated through the **Server Manager** application, once enabled, the **DHCP Manager** application can be used to manage it. The service must be associated with the network interfaces (**binding**) intended to serve. For each network to be served, a **Scope** with an available address **Pool** and other parameters to be provided to clients must be added. It is also possible to set parameters common to all Scopes. The DHCP server on current Windows systems supports both DHCPv4 and DHCPv6.

Linux: There are several free *open source* DHCP servers that support DHCPv4 and DHCPv6, one of the most widely used is the ***ISC DHCP Server*** provided by ISC (Internet Systems Consortium, Inc.). As is usual with Linux / Unix, all *ISC DHCP Server* configuration is done via text files.

DHCPv6 (DHCP for IPv6 clients)

In the case of **IPv6** the DHCPv6 application context is quite different. IPv6 has its own mechanisms for automatic node configuration (*Neighbour Discovery Protocol* – **NDP**) that cannot be bypassed. DHCP may or may not be used depending on IPv6 router configuration.

When an IPv6 node wants to automatically configure the network it sends an ICMPv6 *Router Solicitation* request to link-local multicast address **ff02::2** (*local routers*). If there are IPv6 routers on the network they should respond with the message ICMPv6 ***Router Advertisement***.

The client's behavior is controlled by the router's response, since the response contains the router's address as well as the identification of the IPv6 network addresses it knows. With this information the client gets to know the router and the network it is connected to, so DHCPv6 servers do not need to provide clients with network identification or default gateway.

The answer also contains two bits that determine customer behavior. If the *Managed address configuration* (**M**) bit is set to 1, then the client should use DHCPv6 to know its IPv6 address and complete the information it already knows. Otherwise the client self-assigns an address over the network it already knows using SLAAC.

The bit **O** (*Other configuration*) is only used for **M=0** (SLAAC). If **O=1** DHCPv6 will be used to obtain additional required parameters (not including address already set by SLAAC).

IPv6 Router Advertisement Service

DHCPv6 application depends on *Router Advertisement* responses from **NDP** (*Neighbor Discovery Protocol*), it is a service that must be implemented and configured on IPv6 routers.

Windows and Linux servers can also operate as IPv4 / IPv6 routers, in which case they will have to periodically send to ***Router Advertisements*** networks and also when requested by clients.

Windows Server: the service is integrated into *Routing and Remote Access* (or *Remote Access* only) role. Once installed, the ***Routing and Remote Access*** administrative tool can be used to enable IPv4/IPv6 routing and configure its operation.

Linux: *Router Advertisement Daemon* (**radvd**), open source, can provide this service in Linux routers. In the configuration file **/etc/radvd.conf** for each network interface where the router announces itself, the bit **M** (*AdvManagedFlag*) and the bit **O** (*AdvOtherConfigFlag*) can be set.

DHCP Relay – The DHCP client can only contact the server if it is on the same network (broadcast domain). When using a single DHCP server to serve a large number of sparse networks, this can be inconvenient. DHCP relay service can be installed on a device or server on each network, and configured to relay requests to a remote DHCP server.

ISC DHCP Server (Linux) - Example /etc/dhcpd.conf

```
option domain-name "dei.isep.ipp.pt";
option ip-forwarding off;
option netbios-node-type 8;
option netbios-scope "";
max-lease-time 604800;
default-lease-time 86400;

subnet 100.4.0.0 netmask 255.255.0.0 {
option subnet-mask 255.255.0.0;
option routers 100.4.0.1;
option broadcast-address 100.4.255.255;
option domain-name-servers 192.168.62.15;
option netbios-name-servers 192.168.62.15;
option log-servers 192.168.62.37;
range 100.4.10.0 100.4.30.255;
}
```

The **options** statements correspond to parameters to be provided to clients via DHCP. For each network to serve there must be a **subnet** statement. Declarations made outside a **subnet** block apply to all **subnet** declarations, but may overlap within the block.

Fault tolerant DHCP Service

Because first contact with the server is via *broadcast* (IPv4) or *multicast* (IPv6), implementing a fault-tolerant DHCP service is relatively simple.

If there are multiple DHCP servers on the network (*broadcast* domain or *link-local multicast*), if one of them is not available to respond to clients, others will be.

To avoid conflicts it is necessary to ensure that each server manages a range of different IP addresses of the remaining. Addresses assigned by multiple servers can all belong to the same IP network, or belong to different IP networks, the remaining configuration parameters provided by each DHCP server must be set accordingly. This setting also ensures ***load balancing***, the first server to respond will tend to be the one with the least load.

Microsoft recommends having two DHCP servers, and the available address pool should be split between a primary server with 80% of addresses and a secondary server with 20% of addresses.

For a Windows domain, it is recommended that the DHCP Server role be installed on domain DCs to allow better integration with the *Active Directory* database. In this case, the workstations (*Domain Members*) must be configured to register in their domain DNS their DHCP obtained name.

Name resolution through DNS service (DNS servers)

Among other networking equipment, Windows and Linux servers can perform the DNS server role.

In the case of Windows domains, the role DNS server is part of the various mandatory services that must be present on DCs. When a Windows Server is promoted to *Domain Controller*, role DNS Server is automatically installed. Remember that with Active Directory each Windows domain corresponds to a DNS domain.

In each DNS domain there must be a ***master*** DNS server and at least one ***slave*** DNS server must exist.

The master *server* has the original copy of the DNS records database. *Slave* servers also have a local copy of the database, but get it from the *master*. Both calls themselves ***authoritative name servers*** of the domain because they have local copies of the domain records database. All modifications to the database must be made in the *master*. *Slaves* compare the date of the last database change found in the master (***serial number*** of the SOA register) with the local copy; if they are different they request a total zone transfer (***AFXR DNS query***) or only of changes (incremental) relative to the locally owned *serial number* (***IFXR DNS query***).

After a while unable to synchronize with the *master*, a *slave* server is no longer ***authoritative*** for the domain.

DNS - Zones

Zones are DNS recordsets that belong to the same DNS domain, a DNS server can serve multiple zones. Each zone must have an SOA record that contains the zone master server DNS name, the zone administrator email address (with the **@** symbol replaced by a **dot**), the serial number, and configuration parameters for synchronization of the zone slaves.

Each zone database must also contain zone-related NS records (DNS server names) and subdomain NS records.

Direct zones – contain records used for direct resolution, ie get addresses from qualified names. Notoriously contain type A and AAAA records, respectively to associate names with IPv4 addresses and IPv6 addresses.

Reverse zones – allow ***reverse resolution***, ie obtain the DNS name by providing the IP address. These zone databases correspond to subdomains of the **.in-addr.arpa** special domains. and **ip6.arpa.**, respectively for IPv4 and IPv6 addresses. Each octet in decimal notation (for IPv4) or hexadecimal symbol (for IPv6) corresponds to a subdomain. In both cases the type of register used for this purpose is the PTR (*pointer*).

Note that there is no correspondence between forward zones and reverse zones.

DNS servers

Linux – The most popular DNS server is **bind**, open source, freely available from **ISC** (*Internet Systems Consortium, Inc.*). Its configuration and management of the areas databases in the case of a master is done through configuration files in text format. Also supports changes through client requests (*Dynamic DNS*).

Windows Server – role ***DNS Server*** can be installed on a *Standalone* server, in which case administration should be done manually through the ***DNS Manager*** application.

When a Windows Server is promoted to a domain controller, it is advised to allow the automatic installation of role **DNS Server**, in which case management of the DNS service is automatically performed by *Active Directory Domain Services* (**AD DS**) and manual management is rarely required.

DNS database replication between DCs of a Windows domain does not use master/slave schema. In a Windows domain, usually all DCs are DNS servers, the DNS database is in *Active Directory* and is automatically replicated between the various DCs.

Once the DNS server under the control of AD DS, any machine added to the domain is automatically added to the DNS database.

DNS Server – Linux – Bind version 9

The bind server configuration is based on the **named.conf** file, usually resident in the **/etc/** or **/etc/bind/** folder. Other configuration files are usually included from this file.

One of the most important settings is the definition of ***forwarders***. These are the IP addresses of other DNS servers to which resolution requests can be relayed without using the normal DNS resolution process. Example:

```
options {
        forwarders { 192.249.249.1; 192.249.249.3; };
};
```

Bind can be single or multiple zone *slave*. Statement example:

```
zone "dei.isep.ipp.pt" IN {
        type slave;
        file "/var/lib/bind/dei.isep.ipp.pt";
        masters { 193.136.62.15; };
};
```

Being a slave of the **dei.isep.ipp.pt** zone, the declared zone file /**var/lib/bind/dei.isep.ipp.pt** is created and managed by bind and should not be edited. Contains a copy of the zone database residing on the zone master server (with address **193.136.62.15** in the example).

DNS Server – Linux – Bind version 9

Bind can be single or multiple zone *master*. Statement example:

```
zone "dei.isep.ipp.pt" IN {
        type master;
        file "/etc/bind/direct";
        allow-query { any; };
        allow-transfer { any; };
};
```

In this case it is the zone *master* **dei.isep.ipp.pt**, the declared zone file **/etc/bind/direct** must be created and managed manually..

If bind is configured to support *Dynamic DNS*, it will change the file content accordingly, otherwise the content is only changed manually by the administrator.

The zone file, in this example **/etc/bind/direct**, contains all DNS records that make up the zone database.

In zone files all names must be qualified (**fqdn**) **any name that does not end in period** is considered unqualified and the domain name corresponding to the zone **is automatically added.**

DNS Server – Linux – Bind version 9 – zone file

A zone file is a text file where each line corresponds to an **RR** (*Resource Record*), typically the first record is the **SOA** (*Start Of Authority*). The general format of each zone file line is:

registration-name ttl class registration-type registration-data

registration-name – DNS name registration. Can be omitted, in which case the line must start with a space and the same value as the previous record will be used.

ttl (*Time To Live*) – maximum lifetime in seconds. When a DNS client obtains the registration it should consider that after this time the registration is obsolete and should get a new one. This element is optional, if not specified will be the value from the previous registry or the value declared in the **$TTL** directive.

class – indicates name type, for internet names value is **IN**. As above can be omitted using previous register value.

registration-type – mnemonic which indicates what type of DNS record it is.

registration-data – content of the record, its shape depends on the type of record.

DNS Server – Linux – Bind version 9 – SOA

The **SOA** record is usually the first zone file record and contains the following elements::

master-name administrator-email (serial refresh retry expiry nx)

master-name – DNS name of zone master server.

administrator-email – zone administrator's email address with the **@** sign replaced by a **dot.**

serial – zone serial number. Records the last change to the database in the form **yyyymmddss,** where **yyyymmdd** represents the day it was changed and **ss** the change number on that day (starts with zero).

refresh – number of seconds between *slave* servers checking for updates.

retry – number of seconds *slaves* should wait in case of failure to contact master before retrying.

expiry – number of seconds unable to contact the master after which *slave* servers are no longer authoritative for the zone.

nx – maximum time in seconds of caching for *non-existent domain* (**NXDOMAIN**) responses.

Linux – Bind version 9 – others records

In addition to SOA other important DNS record types are:

Type (mnemonic)	Content
A	IPv4 address
AAAA	IPv6 address
NS	Name server node name (corresponding to an A and/or AAAA record)
CNAME	Node name (corresponding to an A and/or AAAA record)
MX	SMTP server node name and priority (corresponding to an A and/or AAAA record)
PTR	Node name (corresponding to an A and/or AAAA record)
TXT	A comment (also used in implementing SPF*)

* SPF (*Sender Policy Framework*) identify which servers are allowed to send mail on behalf of the domain

PTR records are used in reverse zones, ie corresponding to **in-addr.arpa** and **ip6.arpa.** subdomains, respectively for IPv4 and IPv6 addresses.

Linux – Bind version 9 – example of zone file

```
$TTL 1d
$ORIGIN meu.dominio.pt

@     IN    SOA   nserv.meu.dominio.pt.   admin.meu.dominio.pt ( 2015091502  1d  15M  2W 1h )

              NS          nserv
nserv         A           192.168.70.2
              AAAA        2001:db8:10::2

@             MX          10          mailserv1
              MX          20          mailserv2.meu.dominio.pt.
server1.meu.dominio.pt.               A           192.168.70.90
mailserv1.meu.dominio.pt.             A           192.168.70.23
                                      TXT         "Servidor SMTP principal"
                                      AAAA        2001:db8:10::1
mailserv2                             A           192.168.70.27
mail                  CNAME           mailserv2
www                   CNAME           server1
ftp                   CNAME           server1
```

The **$ORIGIN** statement defines the domain corresponding to the zone, can then be referenced by the @ symbol. All records are of class IN, with 1 day TTL (defined in the **$TTL** statement).